Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине «**Криптография и сетевая безопасность**»
Тема: **«Избыточное кодирование данных в информационных системах. Код Хемминга. Итеративные коды»**

Выполнил:
Студент 4 курса
Группы АС-65
Касьяник К. А.
Проверил:
Хацкевич А. С.

Брест 2026

Цель: приобретение практических навыков кодирования/де- кодирования двоичных данных при использовании кода Хемминга. приобретение практических навыков кодирования/дс- кодирования двоичных данных при использовании итеративных кодов.

Вариант 7

Часть 1

1. На основе информационного сообщения, представленного символами русского/английского алфавитов, служебными символами и цифрами, содержащегося в некоторт текстовом файле, сформировать информационнос сообщение в двоичном виде; длина сообщения в бинарном виде должна быть не менее 16 символов. Для выполнения этого задания можно использовать коды ASCII символов алфавита либо результаты лабораторной работы N3. 2. Для полученного информационного слова проверочную матрицу Хемминга. 3. Используя построенную матрицу, вычислить избыточные символы (слово X_r). 4. Принять исходное слово следующим числом ошибок: 0, 1, 2. Позиция ошибки определяется (генерируется) случайным образом. 5. Для полученного слова $Y_n = Y_k$, Y_r, используя уже известную проверочную матрицу Хемминга, вновь вычислить избыточные символы (обозначим их Y_r), используя выражение (4.6). 6. Вычислить и проанализировать синдром. В случае, если анализ синдрома показал, что информационное сообщение было передано с ошибкой (или 2 ошибками), сгенирировать унарный вектор ошибки $E_n = e_1, e_2, ..., e_n$ и исправить одиночную ошибку, используя формулу (4.7); проанализировать ситуацию при возник- новении ошибки в 2 битах. 7. Результаты оформить в виде шчега по установленным правилам. Файл анализируемой информации (а соответственно и интерфейс приложения) должен содержать исходное информационное сообщение, значения величин k, r, n, проверочную матрицу Хемминга $H_{n,k}$, слово X_n, X_r, Y_n,Y_r, Y_r', синдром S, вектор ошибки E_n. Программа не должна быть чувствительна к длине информационного сообщения.

Часть 2

Разработать собственное приложение, которое позволяет выполнять следующие операции: 1) вписывать произвольное двоичное представление информационного слова X_k (кодируемой информации) длиной k битов в двумерную матрицу размерностью в соответствии с вариантом либо в трехмерную матрицу в соответствии с вариантом (указаны в табл. 5.2); 2) вычислять проверочные биты (биты паритетов): а) по двум; б) по трем; в) по четырем направлениям (группам паритетов); 3) формировать кодовое слово X_n, присоединением избыточных символов к информационному слову; 4) генерировать ошибку произвольной кратности (i, $i > 0$), распределенную случайным (фазом среди символов слова X_n, в результате чего формируется кодовое слово Y_n; 5) определять местоположение ошибочных символов итеративным кодом в слове Y_n в соответствии с исполызусмыми груп-

пами паритетов по пункту (2) и исправлять ошибочные символы (результат исправления слово Y_n'); 6) выполнять анализ корректирующей способности используемого кода (количественная оценка) путем сравнения соответствующих слов X_n, и Y_n' ; результат анализа может быть представлен в виде отношения общего числа сгенерированных кодовых слов с ошибками определенной одинаковой кратности (с одной ошибкой, с двумя ошибками и т. д.) к числу кодовых слов, содержащих ошибки этой кратности, которые правильно обнаружены и которые правильно скорректированы.

Вариант 7

Вариант	Длина информационного слова (бит), k	k_1	k_2	z	Количество групп паритетов
2	20	4	5	–	2; 3
		2	10	–	2; 3
		2	5	2	2; 3; 4; 5
		2	2	5	2; 3; 4; 5

main.py:

```
import random
import sys
from itertools import product


def to_str(data):
    return ''.join(str(x) for x in data)


def convert_text(text):
    result = []
    for byte in text.encode('utf-8'):
        result.extend(int(bit) for bit in f'{byte:08b}')
    return result


def create_hamming_matrix(k):
    r = 0
    while 2 ** r < k + r + 1:
        r += 1

    n = k + r
    matrix = []

    for i in range(r):
        matrix.append([(j >> i) & 1 for j in range(1, n + 1)])

    return r, n, matrix


def get_parity(matrix, data):
    values = []
    for row in matrix:
        values.append(sum(a * b for a, b in zip(row, data)) % 2)
```

```
    return values


def part1(filename):
    with open(filename, 'r', encoding='utf-8') as file:
        bits = convert_text(file.read().strip('\n'))

    k = len(bits)
    if k < 16:
        sys.exit('Сообщение в бинарном виде должно быть не менее 16 бит')

    r, n, matrix = create_hamming_matrix(k)
    positions = {2 ** i - 1 for i in range(r)}

    xn = []
    source = iter(bits)

    for i in range(n):
        xn.append(0 if i in positions else next(source))

    xr = get_parity(matrix, xn)

    for i, pos in enumerate(sorted(positions)):
        xn[pos] = xr[i]

    print('\n' + '=' * 65)
    print('          ЧАСТЬ 1. КОД ХЕММИНГА')
    print('=' * 65)

    print(f'\nИсходные данные:')
    print(f'  Информационное слово : {to_str(bits)}')
    print(f'  Количество бит k     : {k}')
    print(f'  Проверочных бит r    : {r}')
    print(f'  Длина кодового слова : {n}')

    print('\nПроверочная матрица:')
    for i, row in enumerate(matrix, 1):
        print(f'  H[{i}] = {to_str(row)}')

    print('\nФормирование кодового слова:')
    print(f'  Информационные биты : {to_str(bits)}')
    print(f'  Проверочные биты    : {to_str(xr)}')
    print(f'  Кодовое слово Xn    : {to_str(xn)}')

    for errors in range(3):
        en = [0] * n

        for pos in random.sample(range(n), errors):
            en[pos] = 1

        yn = [x ^ e for x, e in zip(xn, en)]
        yr = [yn[pos] for pos in sorted(positions)]

        temp = yn.copy()
        for pos in positions:
            temp[pos] = 0

        yr_new = get_parity(matrix, temp)
        syndrome = [a ^ b for a, b in zip(yr, yr_new)]

        error_pos = sum(bit << i for i, bit in enumerate(syndrome))
```

```
        corrected = yn.copy()
        error_vector = [0] * n

        if 0 < error_pos <= n:
            corrected[error_pos - 1] ^= 1
            error_vector[error_pos - 1] = 1

        print('\n' + '-' * 65)
        print(f'          ПРОВЕРКА ПРИ {errors} ОШИБКАХ')
        print('-' * 65)

        print(f'  Полученное слово Yn  : {to_str(yn)}')
        print(f'  Проверочные биты Yr  : {to_str(yr)}')
        print(f'  Повторный расчёт Yr  : {to_str(yr_new)}')
        print(f'  Синдром              : {to_str(syndrome)}')
        print(f'  Позиция по синдрому  : {error_pos}')
        print(f'  Вектор ошибок En     : {to_str(error_vector)}')

        print('\nРезультат декодирования:')
        print(f'  Исправленное слово   : {to_str(corrected)}')
        print(f'  Исходное слово       : {to_str(xn)}')

        if corrected == xn:
            print('  Результат            : слово восстановлено правильно')
        else:
            print('  Результат            : слово отличается от исходного')

    print('\n' + '=' * 65)
    print('             ЗАВЕРШЕНИЕ ПЕРВОЙ ЧАСТИ')
    print('=' * 65)


DIRECTIONS_2D = [
    (0, 1),
    (1, 0),
    (1, 1),
    (1, -1)
]

DIRECTIONS_3D = [
    (0, 1, 0),
    (1, 0, 0),
    (0, 0, 1),
    (1, 1, 1),
    (1, -1, 1)
]


def generate_lines(dimensions, directions):
    coordinates = list(product(*(range(size) for size in dimensions)))
    indexes = {point: i for i, point in enumerate(coordinates)}
    result = []

    for direction in directions:
        for point in coordinates:
            previous = tuple(x - d for x, d in zip(point, direction))

            if previous in indexes:
                continue

            current = point
            line = []
```

```
            while current in indexes:
                line.append(indexes[current])
                current = tuple(x + d for x, d in zip(current, direction))

            result.append(line)

    return result


def make_codeword(data, lines):
    checks = []

    for line in lines:
        checks.append(sum(data[i] for i in line) % 2)

    return data + checks

def correct_errors(word, k, lines, cell_lines):
    result = word.copy()
    detected = False
    for _ in range(len(result)):
        invalid = set()
        for j, line in enumerate(lines):
            if (result[k + j] + sum(result[i] for i in line)) % 2:
                invalid.add(j)

        if not invalid:
            break

        detected = True
        if len(invalid) == 1:
            index = k + next(iter(invalid))
            result[index] ^= 1
            continue

        candidates = [
            i for i in range(k)
            if cell_lines[i].issubset(invalid)
        ]
        if len(candidates) != 1:
            break

        result[candidates[0]] ^= 1

    return result, detected

def part2(k=20, trials=10000, max_errors=6):
    configurations = [
        ((4, 5), (2, 3)),
        ((2, 10), (2, 3)),
        ((2, 5, 2), (2, 3, 4, 5)),
        ((2, 2, 5), (2, 3, 4, 5))
    ]
    print('\n' + '=' * 65)
    print('       ЧАСТЬ 2. ИТЕРАТИВНОЕ КОДИРОВАНИЕ')
    print('=' * 65)
    print(f'\nДлина информационного слова: {k} бит')
    print(f'Количество испытаний       : {trials}')
    print(f'Максимальное число ошибок  : {max_errors}')
    for dimensions, groups in configurations:
        print('\n' + '#' * 65)
```

```
        print(f'РАЗМЕРНОСТЬ МАТРИЦЫ: {" x ".join(map(str, dimensions))}')
        print('#' * 65)

        for group_count in groups:
            directions = DIRECTIONS_2D if len(dimensions) == 2 else DIRECTIONS_3D
            lines = generate_lines(dimensions, directions[:group_count])
            cell_lines = []
            for i in range(k):
                related = {
                    j for j, line in enumerate(lines)
                    if i in line
                }
                cell_lines.append(related)

            xk = [random.randrange(2) for _ in range(k)]
            xn = make_codeword(xk, lines)
            n = len(xn)
            print(f'\nГрупп паритетов: {group_count}')
            print(f'  Информационное слово : {to_str(xk)}')
            print(f'  Кодовое слово Xn     : {to_str(xn)}')
            print(f'  Размер кодового слова: {n} бит')
            print('\n  Результаты моделирования:')
            print('  ' + '-' * 59)
            print(f'  {"Ошибок":<10} {"Обнаружено":<17} {"Исправлено":<17}')
            print('  ' + '-' * 59)

            for error_count in range(1, min(max_errors, n) + 1):
                detected_count = 0
                corrected_count = 0
                for _ in range(trials):
                    yn = xn.copy()
                    for pos in random.sample(range(n), error_count):
                        yn[pos] ^= 1

                    result, detected = correct_errors(
                        yn, k, lines, cell_lines
                    )
                    detected_count += detected
                    corrected_count += result == xn

                detected_percent = detected_count / trials * 100
                corrected_percent = corrected_count / trials * 100
                print(
                    f'  {error_count:<10} '
                    f'{detected_percent:>6.2f}% ({detected_count:<5}) '
                    f'{corrected_percent:>6.2f}% ({corrected_count:<5})'
                )

            print('  ' + '-' * 59)

    print('\n' + '=' * 65)
    print('              ЗАВЕРШЕНИЕ ВТОРОЙ ЧАСТИ')
    print('=' * 65)

if __name__ == '__main__':
    print('============================================================ p1')
    part1(sys.argv[1] if len(sys.argv) > 1 else 'input.txt')

    print('\n============================================================ p2')
    part2()
```

```
================================================================
          ЧАСТЬ 1. КОД ХЕММИНГА
================================================================

Исходные данные:
  Информационное слово : 011000110110000101110100
  Количество бит k     : 24
  Проверочных бит r    : 5
  Длина кодового слова : 29

Проверочная матрица:
  H[1] = 10101010101010101010101010101
  H[2] = 01100110011001100110011001100
  H[3] = 00011110000111100001111000011
  H[4] = 00000001111111100000000111111
  H[5] = 00000000000000011111111111111

Формирование кодового слова:
  Информационные биты : 011000110110000101110100
  Проверочные биты    : 10111
  Кодовое слово Xn    : 100111010011011100000101110100

----------------------------------------------------------------
          ПРОВЕРКА ПРИ 0 ОШИБКАХ
----------------------------------------------------------------
  Полученное слово Yn  : 100111010011011100000101110100
  Проверочные биты Yr  : 10111
  Повторный расчёт Yr  : 10111
  Синдром              : 00000
  Позиция по синдрому  : 0
  Вектор ошибок En     : 00000000000000000000000000000

Результат декодирования:
  Исправленное слово   : 100111010011011100000101110100
  Исходное слово       : 100111010011011100000101110100
  Результат            : слово восстановлено правильно

----------------------------------------------------------------
          ПРОВЕРКА ПРИ 1 ОШИБКАХ
----------------------------------------------------------------
  Полученное слово Yn  : 100111010011011100000100110100
  Проверочные биты Yr  : 10111
  Повторный расчёт Yr  : 01010
  Синдром              : 11101
  Позиция по синдрому  : 23
  Вектор ошибок En     : 00000000000000000000001000000

Результат декодирования:
  Исправленное слово   : 100111010011011100000101110100
  Исходное слово       : 100111010011011100000101110100
  Результат            : слово восстановлено правильно

----------------------------------------------------------------
          ПРОВЕРКА ПРИ 2 ОШИБКАХ
----------------------------------------------------------------
  Полученное слово Yn  : 100111010001011100000101110110
  Проверочные биты Yr  : 10111
  Повторный расчёт Yr  : 01010
  Синдром              : 11101
  Позиция по синдрому  : 23
  Вектор ошибок En     : 00000000000000000000001000000

Результат декодирования:
  Исправленное слово   : 100111010001011100000100110110
  Исходное слово       : 100111010011011100000101110100
  Результат            : слово отличается от исходного
```

```
================================================================
       ЧАСТЬ 2. ИТЕРАТИВНОЕ КОДИРОВАНИЕ
================================================================

Длина информационного слова: 20 бит
Количество испытаний       : 10000
Максимальное число ошибок  : 6

################################################################################
РАЗМЕРНОСТЬ МАТРИЦЫ: 4 x 5
################################################################################

Групп паритетов: 2
  Информационное слово : 10000011011101101101
  Кодовое слово Xn     : 10000011011101101101110101011
  Размер кодового слова: 29 бит

  Результаты моделирования:
  ------------------------------------------------------------
  Ошибок     Обнаружено        Исправлено
  ------------------------------------------------------------
  1          100.00% (10000) 100.00% (10000)
  2          100.00% (10000)   0.00% (0    )
  3           99.48% (9948 )   0.00% (0    )
  4           99.39% (9939 )   0.00% (0    )
  5           99.75% (9975 )   0.00% (0    )
  6           99.73% (9973 )   0.00% (0    )
  ------------------------------------------------------------

Групп паритетов: 3
  Информационное слово : 01000001000101111010
  Кодовое слово Xn     : 0100000100010111101011111110111100111
  Размер кодового слова: 37 бит

  Результаты моделирования:
  ------------------------------------------------------------
  Ошибок     Обнаружено        Исправлено
  ------------------------------------------------------------
  1          100.00% (10000) 100.00% (10000)
  2          100.00% (10000)  42.07% (4207 )
  3          100.00% (10000)   0.00% (0    )
  4           99.96% (9996 )   0.00% (0    )
  5          100.00% (10000)   0.00% (0    )
  6          100.00% (10000)   0.00% (0    )
  ------------------------------------------------------------

################################################################################
РАЗМЕРНОСТЬ МАТРИЦЫ: 2 x 10
################################################################################

Групп паритетов: 2
  Информационное слово : 10110011011111100010
  Кодовое слово Xn     : 10110011011111100010000100101111
  Размер кодового слова: 32 бит

  Результаты моделирования:
  ------------------------------------------------------------
  Ошибок     Обнаружено        Исправлено
  ------------------------------------------------------------
  1          100.00% (10000) 100.00% (10000)
  2          100.00% (10000)   0.00% (0    )
  3           99.54% (9954 )   0.00% (0    )
  4           99.66% (9966 )   0.00% (0    )
  5           99.94% (9994 )   0.00% (0    )
  6           99.83% (9983 )   0.00% (0    )
  ------------------------------------------------------------
```

```
Групп паритетов: 3
  Информационное слово : 00001110010010111010
  Кодовое слово Xn     : 0000111001001011101001001000001101010011010
  Размер кодового слова: 43 бит

  Результаты моделирования:
  ------------------------------------------------------------
  Ошибок     Обнаружено        Исправлено
  ------------------------------------------------------------
  1          100.00% (10000) 100.00% (10000)
  2          100.00% (10000)  44.23% (4423 )
  3          100.00% (10000)   0.00% (0    )
  4           99.99% (9999 )   0.00% (0    )
  5          100.00% (10000)   0.00% (0    )
  6           99.99% (9999 )   0.00% (0    )
  ------------------------------------------------------------

##################################################################
РАЗМЕРНОСТЬ МАТРИЦЫ: 2 x 5 x 2
##################################################################

Групп паритетов: 2
  Информационное слово : 10111111011111001000
  Кодовое слово Xn     : 1011111101111100100000100100110101
  Размер кодового слова: 34 бит

  Результаты моделирования:
  ------------------------------------------------------------
  Ошибок     Обнаружено        Исправлено
  ------------------------------------------------------------
  1          100.00% (10000) 100.00% (10000)
  2          100.00% (10000)  25.00% (2500 )
  3           99.56% (9956 )   0.00% (0    )
  4           99.84% (9984 )   0.00% (0    )
  5           99.97% (9997 )   0.00% (0    )
  6           99.95% (9995 )   0.00% (0    )
  ------------------------------------------------------------

Групп паритетов: 3
  Информационное слово : 01000011010110000110
  Кодовое слово Xn     : 0100001101011000011011000010001011100011101
  Размер кодового слова: 44 бит

  Результаты моделирования:
  ------------------------------------------------------------
  Ошибок     Обнаружено        Исправлено
  ------------------------------------------------------------
  1          100.00% (10000) 100.00% (10000)
  2          100.00% (10000)  44.24% (4424 )
  3          100.00% (10000)   0.00% (0    )
  4           99.98% (9998 )   0.00% (0    )
  5          100.00% (10000)   0.00% (0    )
  6           99.99% (9999 )   0.00% (0    )
```

```
Групп паритетов: 4
  Информационное слово : 0101111110110101000
  Кодовое слово Xn     : 010111111101101010001111001101011111000111100101111111011110
  Размер кодового слова: 60 бит

  Результаты моделирования:
  ------------------------------------------------------------
  Ошибок     Обнаружено        Исправлено
  ------------------------------------------------------------
  1          100.00% (10000) 100.00% (10000)
  2          100.00% (10000)  39.38% (3938 )
  3          100.00% (10000)   0.00% (0    )
  4          100.00% (10000)   0.00% (0    )
  5          100.00% (10000)   0.00% (0    )
  6          100.00% (10000)   0.00% (0    )
  ------------------------------------------------------------

Групп паритетов: 5
  Информационное слово : 01010111010011100010
  Кодовое слово Xn     : 0101011101001110001011110110111111111101001011101011101001101010111110101101010
  Размер кодового слова: 76 бит

  Результаты моделирования:
  ------------------------------------------------------------
  Ошибок     Обнаружено        Исправлено
  ------------------------------------------------------------
  1          100.00% (10000) 100.00% (10000)
  2          100.00% (10000)  35.65% (3565 )
  3          100.00% (10000)   0.00% (0    )
  4          100.00% (10000)   0.00% (0    )
  5          100.00% (10000)   0.00% (0    )
  6          100.00% (10000)   0.00% (0    )
  ------------------------------------------------------------

####################################################################
РАЗМЕРНОСТЬ МАТРИЦЫ: 2 x 2 x 5
####################################################################

Групп паритетов: 2
  Информационное слово : 11000111010000100010
  Кодовое слово Xn     : 1100011101000010001000101000111100111111
  Размер кодового слова: 40 бит

  Результаты моделирования:
  ------------------------------------------------------------
  Ошибок     Обнаружено        Исправлено
  ------------------------------------------------------------
  1          100.00% (10000) 100.00% (10000)
  2          100.00% (10000)  40.98% (4098 )
  3           99.81% (9981 )   0.00% (0    )
  4           99.98% (9998 )   0.00% (0    )
  5          100.00% (10000)   0.00% (0    )
  6          100.00% (10000)   0.00% (0    )
```

```
Групп паритетов: 3
  Информационное слово : 01111000010011100100
  Кодовое слово Xn     : 011110000100111001000111000011010000001010111
  Размер кодового слова: 44 бит

  Результаты моделирования:
  ------------------------------------------------------------
  Ошибок     Обнаружено        Исправлено
  ------------------------------------------------------------
  1          100.00% (10000) 100.00% (10000)
  2          100.00% (10000)  43.82% (4382 )
  3          100.00% (10000)   0.00% (0    )
  4          100.00% (10000)   0.00% (0    )
  5          100.00% (10000)   0.00% (0    )
  6          100.00% (10000)   0.00% (0    )
  ------------------------------------------------------------

Групп паритетов: 4
  Информационное слово : 10011001110110010011
  Кодовое слово Xn     : 100110011101100100111010011111111111010011011010100111011001
  Размер кодового слова: 60 бит

  Результаты моделирования:
  ------------------------------------------------------------
  Ошибок     Обнаружено        Исправлено
  ------------------------------------------------------------
  1          100.00% (10000) 100.00% (10000)
  2          100.00% (10000)  41.80% (4180 )
  3          100.00% (10000)   0.00% (0    )
  4          100.00% (10000)   0.00% (0    )
  5          100.00% (10000)   0.00% (0    )
  6          100.00% (10000)   0.00% (0    )
  ------------------------------------------------------------

Групп паритетов: 5
  Информационное слово : 11011110111000111
  Кодовое слово Xn     : 1101111011100011111100000011100101000100000100101110111000111101111001111111
  Размер кодового слова: 76 бит

  Результаты моделирования:
  ------------------------------------------------------------
  Ошибок     Обнаружено        Исправлено
  ------------------------------------------------------------
  1          100.00% (10000) 100.00% (10000)
  2          100.00% (10000)  35.36% (3536 )
  3          100.00% (10000)   0.00% (0    )
  4          100.00% (10000)   0.00% (0    )
  5          100.00% (10000)   0.00% (0    )
  6          100.00% (10000)   0.00% (0    )
```

При возникновении двух ошибок синдром вычисляется как XOR позиций p_1 и p_2. Полученное значение не равно нулю, поэтому оно может быть воспринято как позиция одиночной ошибки в другом бите. Таким образом, стандартный код Хэмминга (с минимальным расстоянием 3) исправляет одиночные ошибки, но не способен корректно распознать двойную и может выполнить неправильное исправление.

Если результат p_1 XOR p_2 превышает n, такой позиции в кодовом слове нет, поэтому программа оставляет его без изменений (условие if pos <= n). Для обнаружения двойных ошибок и их отличия от одиночных используется расширенный код Хэмминга с дополнительным общим битом чётности.

Вывод: приобрел практических навыков кодирования/де- кодирования двоичных данных при использовании кода Хемминга. приобретел практических навыков кодирования/дс- кодирования двоичных данных при использовании итеративных кодов.